

REXEL-Based Knowledge Graph Extraction Pipeline

Method, Implementation, and Results Report

Bibek Singh Dhody

Roll No: 2023101054

May 8, 2026

Abstract

This report presents an end-to-end knowledge graph extraction pipeline that converts noisy natural language documents into structured graph data. The system combines text denoising, phonetic correction, joint neural extraction using a REXEL-style architecture, optional LLM-based triple verification (GraphJudge), and graph persistence in JSON and Neo4j formats. The core model jointly predicts mentions, entity clusters, and relations from a shared contextual encoder. Experiments on the Luke Skywalker benchmark example indicate that extraction quality is highly sensitive to verification settings: strict verification provides fewer but higher-confidence edges, while disabling verification maximizes graph density and recall. The implementation supports practical deployment modes for exploration, balanced filtering, and precision-first curation.

1 Introduction

Knowledge graph (KG) construction from long and noisy text is a central problem in information extraction. A practical KG pipeline must handle spelling artifacts, alias variation, ambiguity in entity mentions, and inconsistencies in relation phrasing. Traditional modular pipelines often separate named entity recognition, coreference resolution, and relation extraction into independent stages. While modularity is convenient, this separation can propagate errors and reduce consistency across outputs.

The present system follows a joint extraction strategy centered around a REXEL-style document-level model. The project objective is straightforward: transform unstructured text into graph-ready triples, where entities are nodes and relations are typed edges. The implementation further includes optional post-extraction verification with a language model, followed by storage in both local JSON format and a Neo4j graph database. The report content is based on the updated project README available on the `temp-main` branch of the project repository: <https://github.com/bibesgotthevibes/btp.git> (branch: `temp-main`).

2 System Overview and Pipeline Flow

2.1 End-to-End Runtime Sequence

The pipeline executes the following ordered stages:

1. `denoise_text`
2. `correct_phonetics`
3. `rexel_extract`
4. `judge_triples`
5. `store_to_neo4j`

Operationally, the system accepts either raw text input or a file path, generates draft triples through REXEL extraction, optionally filters those triples through GraphJudge, and writes finalized output to `kg_output.json` and (when configured) to Neo4j.

2.2 Text Denoising

The `denoise_text` stage applies light-weight document cleaning to remove OCR/noise artifacts and normalize whitespace and punctuation. Implementation heuristics strip repeated non-alphanumeric tokens, collapse excessive whitespace, and perform conservative sentence segmentation to preserve textual context for downstream span detection.

2.3 Phonetic Correction

The `correct_phonetics` stage applies phonetic-candidate correction to recover probable surface forms (e.g., entity name spelling errors). The pipeline uses a phonetic matching approach combined with a small candidate-ranking heuristic: generate candidate corrections using phonetic similarity, rank by local lexical context and token frequency, then apply top-ranked substitutions where confidence is high. This reduces fragmentation of entity mentions prior to joint extraction.

2.4 Design Motivation

The chosen architecture balances robustness and practical usability:

- Upstream text conditioning improves downstream extraction quality.
- Joint modeling captures interactions between span detection, entity typing, coreference, and relation prediction.
- Optional judge mode supports configurable precision-recall behavior.
- Dual output (JSON + Neo4j) supports both experimentation and graph analytics workflows.

3 Core Extraction Method: REXEL

The extraction backbone is implemented as a joint document information extraction model with a shared encoder and multiple task-specific heads.

3.1 Shared Encoder

The model uses RoBERTa-base to generate contextual token representations over the input document. This common representation space supports all downstream extraction sub-tasks and encourages shared semantics between mention, entity, and relation predictions.

3.2 Joint Prediction Heads

The implemented heads include:

- **Span Head:** Scores candidate mention spans.
- **Type Head:** Predicts entity type labels.
- **Relation Head:** Predicts relation labels for ordered entity pairs.
- **Coref Head:** Estimates coreference compatibility across mentions/entities.
- **Link Projector:** Provides projection support for entity linking compatibility.

This multi-head setup enables document-level consistency and reduces fragmentation that typically appears in loosely coupled extraction pipelines.

3.3 Mention Proposal and Filtering Strategy

Candidate spans are enumerated up to bounded width (capped at 4 during inference), scored, and filtered. Filtering rules remove cross-sentence spans, punctuation-heavy spans, and clause-like segments with interior stopwords. A final overlap-suppression phase keeps high-confidence mentions and removes weak fragments.

3.4 Entity Clustering and Coreference Merge

Entity consolidation follows a two-stage merge policy:

1. String normalization grouping for case and punctuation variants.
2. Neural coreference merge with a high threshold and token-sharing safeguards.

This hybrid approach improves canonicalization while reducing alias duplication and relation inconsistency.

3.5 Relation Extraction and Triple Finalization

For each ordered entity pair, relation logits are computed and converted to confidence scores. Non-NA labels above the relation threshold are retained and duplicate triples are removed. The method favors recall during candidate generation and allows optional downstream filtering.

3.6 Optional Entity Linking Hook

A non-blocking hook supports Wikidata enrichment when linking resources are available. If linking is unavailable, extraction still proceeds without pipeline failure.

4 GraphJudge Verification Layer

4.1 Purpose

GraphJudge is an optional LLM-based verification stage used to validate extracted triples using textual evidence. It is designed as a configurable post-processing layer rather than a mandatory component.

4.2 Implemented Enhancements

The current implementation includes several practical upgrades:

- Mapping from Wikidata properties (P-codes) to readable relation labels.
- Evidence-focused context windows instead of full-passage prompting.
- Alias-aware matching using canonical names and cluster mentions.
- Backward compatibility for both `is_true` and `valid` JSON output schemas.
- Strictness control via runtime flags.

4.3 Modes of Operation

- **Strict mode** (`JUDGE_STRICT=true`): rejects uncertain triples; precision-oriented.
- **Non-strict mode** (`JUDGE_STRICT=false`): keeps undecidable triples and can recover false negatives using lexical co-occurrence fallback.
- **Bypass mode** (`JUDGE_ENABLED=false`): no verification; all REXEL triples retained.

Empirically, the judge stage is currently conservative and can reduce recall substantially in strict configurations.

5 Storage and Graph Materialization

Final outputs are written in two forms:

- **JSON Output:** `kg_output.json` (always generated)
- **Neo4j Output:** entity nodes and typed relationships (generated when connection succeeds)

The Neo4j writer supports both triple schemas:

- `subject/predicate/object`
- `head/relation/tail`

This dual-schema compatibility addresses prior failure modes where database connectivity was successful but edges were not materialized due to schema mismatch.

6 Experimental Setup

6.1 Input and Checkpoint

Experiments used:

- Input file: `data/example_docs/luke_skywalker.txt`
- Best checkpoint: `checkpoints/rexel_joint_v8_best.pt`
- REXEL model (checkpoint archive): <https://drive.google.com/file/d/1hab1pDJG1M4PBiEq5BjYdj2Ssy4fz1Y6/view?usp=sharing>

6.2 Inference Configuration

The reported runs used the following thresholds:

- Span threshold: 0.10
- Relation threshold: 0.05

Three judge settings were evaluated under these otherwise consistent conditions.

7 Results

7.1 Quantitative Summary

Mode	Entities	Draft Triples	Final Edges	Judge Configuration
Strict Judge ON	29	26	5	JUDGE_ENABLED=true, JUDGE_STRICT=true
Strict Judge OFF	26	24	9	JUDGE_ENABLED=true, JUDGE_STRICT=false
Judge Disabled	25	20	20	JUDGE_ENABLED=false

Table 1: Observed extraction outcomes on the Luke Skywalker example.

7.2 Interpretation

The strongest pattern is a clear precision-recall trade-off induced by the judge stage:

- Strict judge mode yields the sparsest graph, prioritizing confidence over coverage.
- Non-strict mode recovers additional edges and performs better than prior judge settings due to improved evidence selection and lexical-support fallback.
- Disabling judge yields maximum graph density and highest edge retention, suitable for exploratory analysis.

8 Error Analysis and Practical Observations

8.1 Primary Bottleneck

The judge layer is currently the dominant bottleneck for edge retention. In practical workloads, this can suppress many semantically plausible triples when evidence is ambiguous or phrased indirectly.

8.2 Typical Rejection Patterns

Observed causes include:

- Canonicalization artifacts (name variants and punctuation fragmentation).
- Conservative LLM validation despite partial textual support.
- Semantic mismatch between extracted relation labels and passage wording.

8.3 Mitigations Already Incorporated

The implementation already includes several mitigations:

- Relation hinting through PID label expansion.
- Evidence-focused context extraction around entity aliases.
- Alias-aware verification using cluster mentions.
- Backward-compatible output parsing.
- Non-strict lexical fallback.
- Configurable judge bypass for recall-first operation.

8.4 Variance Across Runs

Minor run-to-run variation can occur due to LLM-driven denoising and phonetic correction, which alter upstream text and indirectly change extraction outcomes.

9 Recommended Deployment Modes

Based on current behavior, the following operating profiles are recommended:

- **Demo / Exploration:** JUDGE_ENABLED=false
- **Balanced Filtering:** JUDGE_ENABLED=true, JUDGE_STRICT=false
- **Precision-First Curation:** JUDGE_ENABLED=true, JUDGE_STRICT=true

Note: JUDGE_STRICT has effect only when JUDGE_ENABLED=true.

10 Reproducibility Command

A representative command from the project is shown below:

```
python -m kg_pipeline.main \  
  --input-file data/example_docs/luke_skywalker.txt \  
  --rexel-backend torch \  
  --rexel-checkpoint checkpoints/rexel_joint_v8_best.pt \  
  --rexel-span-threshold 0.10 \  
  --rexel-relation-threshold 0.05
```

11 Conclusion

The project delivers a complete, operational KG extraction pipeline with practical storage and visualization support. The REXEL extraction backbone is effective in producing structured graph candidates from noisy text, and Neo4j integration enables direct graph materialization for downstream querying and analysis. The optional GraphJudge layer introduces valuable curation capability but remains the main recall bottleneck in strict mode. For high-coverage exploratory workflows, judge bypass currently provides the most useful graph density. For curated outputs, strict verification remains appropriate when lower recall is acceptable.

Individual Contribution

Bibek Singh Dhody - Implemented upstream preprocessing (denoise and phonetic correction), developed the GraphJudge verification layer, and integrated the pipeline components prior to REXEL.

Sanyam Agarwal - Implemented the REXEL joint extraction model, performed model training and check-pointing, and implemented Neo4j graph materialization.